

KUBERNETES IN PRODUCTION · ISSUE 04 OF 08

Debugging CrashLoopBackOff and ImagePullBackOff.

Read the exit code and the events before you read the code.

04

AR

Akhilesh Ranjan Singh

Senior DevSecOps Engineer · github.com/iarsingh

Swipe →

Triage order

1 get pods

2 describe pod

3 logs --previous

4 get events

5 debug container

Most issues are solved by step three.

The commands

```
kubectl get pods -n app
kubectl describe pod api-7d9 -n app
kubectl logs api-7d9 -n app --previous
kubectl get events -n app \
  --sort-by=.lastTimestamp
kubectl debug -it api-7d9 -n app \
  --image=busybox --target=api
```

--previous shows the logs of the crashed container, not the new one.

CrashLoopBackOff causes

SIGNAL	LIKELY CAUSE
Exit 1	App error — read previous logs
Exit 137 / OOMKilled	Memory limit too low or leak
Exit 127	Command or binary not found
Liveness failures in events	Probe too strict
Crash on boot	Missing env, config or secret

ImagePullBackOff causes

01 Tag does not exist

Typo, or the build never pushed.

02 No registry auth

Missing imagePullSecret or workload identity.

03 No route to registry

Private nodes without NAT or private endpoint.

04 Platform mismatch

arm64-only image on amd64 nodes.

After you fix it

- ☒ Add the case to the runbook

- ☒ Alert on restart count

- ☒ Pin images by digest

- ☒ Validate config in CI

- ☒ Test the image on the node arch



KEY TAKEAWAY

Exit code, previous logs, events. In that order.

- CrashLoopBackOff is a symptom
- 137 means memory
- Pull errors are usually auth or network

YOUR TURN

What is the strangest root cause you found behind a CrashLoopBackOff?

FOLLOW AKHILESH RANJAN SINGH

Next in the series: #05 Namespaces and RBAC for multi-team clusters

Save · Repost

